

From Nand to Tetris

NPU Extension

Chapter 4: 模型训练与量化导出

Model Training, Quantization and Export

Project

Building a Modern Computer From First Principles

Background

硬件已经具备计算卷积和全连接的能力，但还没有权重。本章用 PyTorch 训练一个很小的 MNIST 分类模型，做后训练量化，并把权重和 golden 输出导出成 RTL 能加载的格式。

这是“软件定义硬件数据”的环节：模型结构、每层 scale、导出布局都会直接影响 RTL 控制器。

Objective

训练一个 tiny CNN（约 1-5 万参数），MNIST 准确率不低于 95%。

实现 int8 后训练量化：权重、激活都映射到 $[-128, 127]$ 。

编写 `tools/npu_mnist.py`，导出 `weights.hex`、`image.hex`、`golden.hex`、`scales.json`。

用 Python int8 参考模拟器验证导出的权重和 golden。

Deliverables

- `tools/npu_mnist.py`：训练、量化、导出一体脚本。
- `npu_data/`：权重、图片、golden、scale 文件。
- 量化精度报告：float 模型 vs int8 模拟的准确率对比。

Contract

- 导出格式必须与 Ch3 定义的 SRAM 布局一致（行主序、int8 补码）。
- 同一份权重在 Python int8 模拟与 RTL 中必须得到相同结果。
- 脚本可重复：固定随机种子，`python3 tools/npu_mnist.py --all` 一键完成。

Resources

- 本机已安装 PyTorch 2.13 与 torchvision 0.28。
- `docs/npu.md` 中的模型结构建议。
- Ch3 的 golden 格式约定。

Tips

- 先训练一个浮点小模型，再量化；不要一开始就追求大模型。
- 量化最简单的方式：per-tensor min/max $\rightarrow$ scale = range/255。
- 把 BN 折叠进卷积权重，减少 RTL 工作量。
- golden 用整数运算生成：round 规则必须与 RTL 一致（四舍五入）。